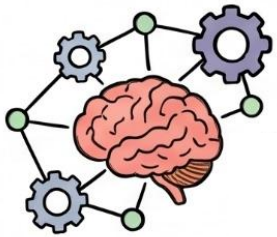
UNIVERSIDAD DEL MAR
Campus Huatulco

PRIMERA JORNADA ACADÉMICA DE ACTUARÍA
MÁS ALLÁ DEL AULA

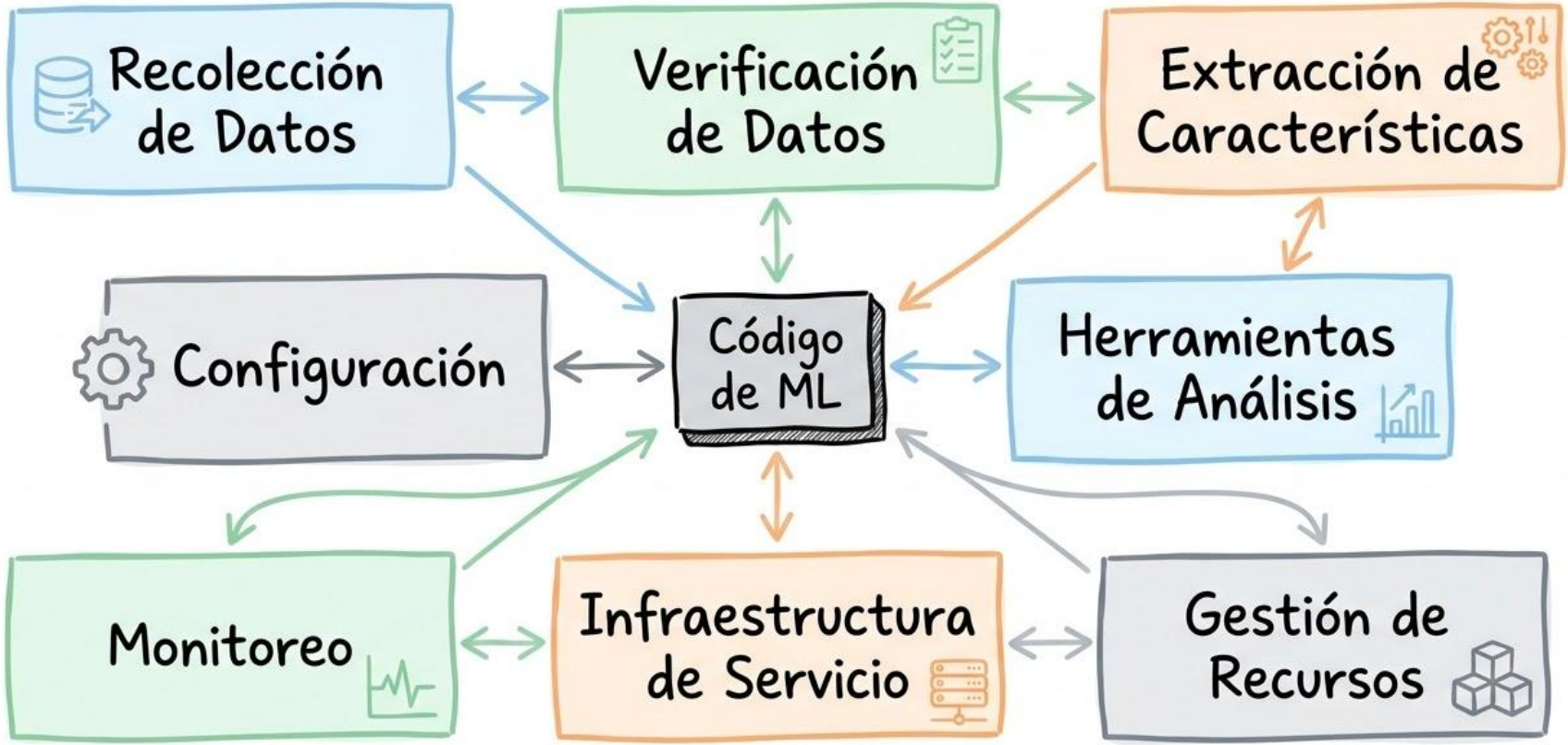


EL CICLO DE VIDA DE UN MODELO DE ML



Ponente:
Dra. Claudia Ivonne Juárez Gallegos

Deuda Técnica en Sistemas de Aprendizaje Automático (ML)

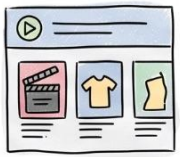


Capacidades de ML / IA

Casos de Uso de ML

Capacidades de ML/IA en Producción

Recomendación



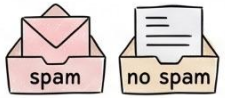
Identifica el producto relevante para el usuario

Selección Top-K



Organiza un conjunto de elementos

Clasificación



Asigna ejemplos a clases predefinidas

Generación de Contenido



Produce nuevo contenido aprendiendo de ejemplos

Respuestas a Preguntas



Responde a una pregunta explícita

Automatización





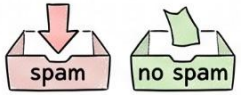
Realiza automáticamente un conjunto de pasos del usuario

Detección de Fraude y Anomalías



Identifica acciones o transacciones sospechosas

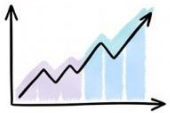
Clasificación



Clasificación

Asigna ejemplos a clases predefinidas

Predicción



Asigna el valor más probable a una entidad

Generación de Contenido o imagen



Produce nuevo contenido aprendiendo de ejemplos

Extracción de Información y Anotación

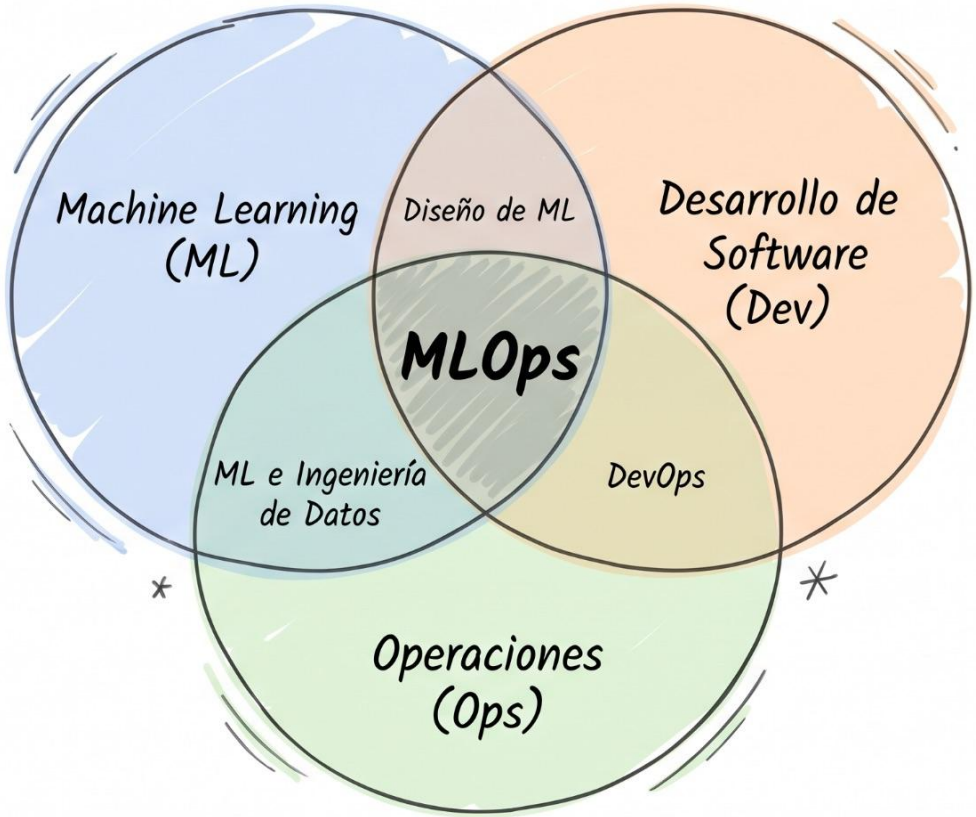


Identifica datos clave en textos (nombres, ubicaciones)

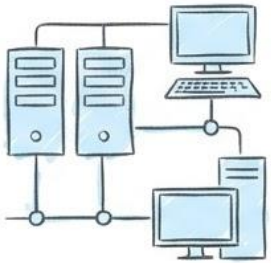
La Brecha del Despliegue (Deployment Gap)

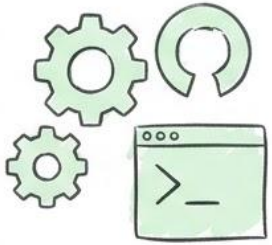
El verdadero valor de estas capacidades solo se logra al desplegarlas en sistemas reales.

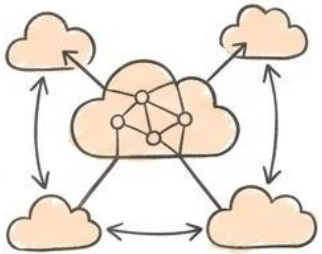
¿Qué es MLOps?

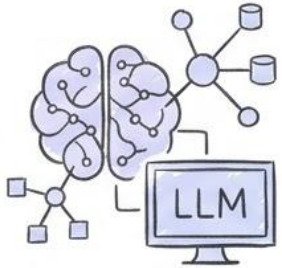

Machine Learning (ML)
Diseño de ML
Desarrollo de Software (Dev)
MLOps
ML e Ingeniería de Datos
DevOps
Operaciones (Ops)

Evolución de MLOps y Plataformas de ML (2015-2026)









2015 - Sistemas Internos

Plataformas de grandes empresas para necesidades específicas

- Google TFX
- Uber Michelangelo
- Airbnb Bighead

2018 - Herramientas Open Source

Democratización y modularidad con el código abierto

- MLflow
- Kubeflow
- DVC

2020-2022 - Plataformas Cloud End-to-End

La nube centraliza todo el ciclo de vida

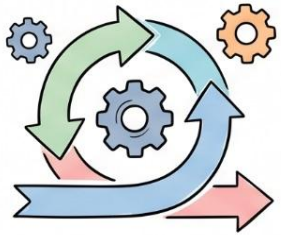
- AWS SageMaker
- GCP Vertex AI
- Azure ML

2024-2026 - Era Moderna y LLMOps

Modelos de lenguaje, flujos rápidos y monitoreo constante

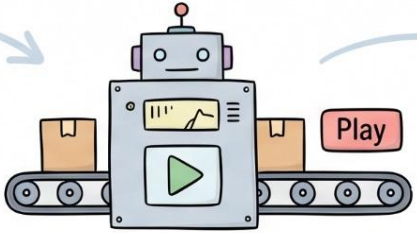
- Databricks
- LangChain
- Monitoreo Continuo

Los 7 Principios de MLOps (ml-ops.org)



Iterativo-Incremental
(Espiral de desarrollo)

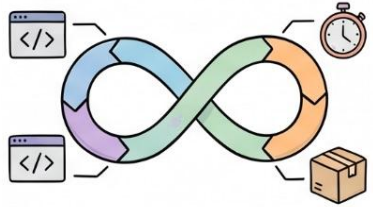
El ciclo de vida se recorre una y otra vez (Diseño ➜ Desarrollo ➜ Operaciones) para mejorar constantemente.



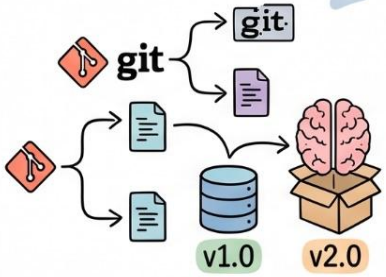
Automatización
(Eliminar procesos manuales)

Automatizar la preparación de datos, el entrenamiento y el despliegue sin intervención manual.

Continuo
(CI / CD / CT / CM)



Integración, Entrega, Entrenamiento y Monitoreo constantes en producción.



Versioning
(Control de versiones)

No solo versionar código; también **versionar** datos, hiperparámetros y modelos.



Pruebas (Testing)

Validar la **calidad** de los datos, el código y las predicciones del modelo.



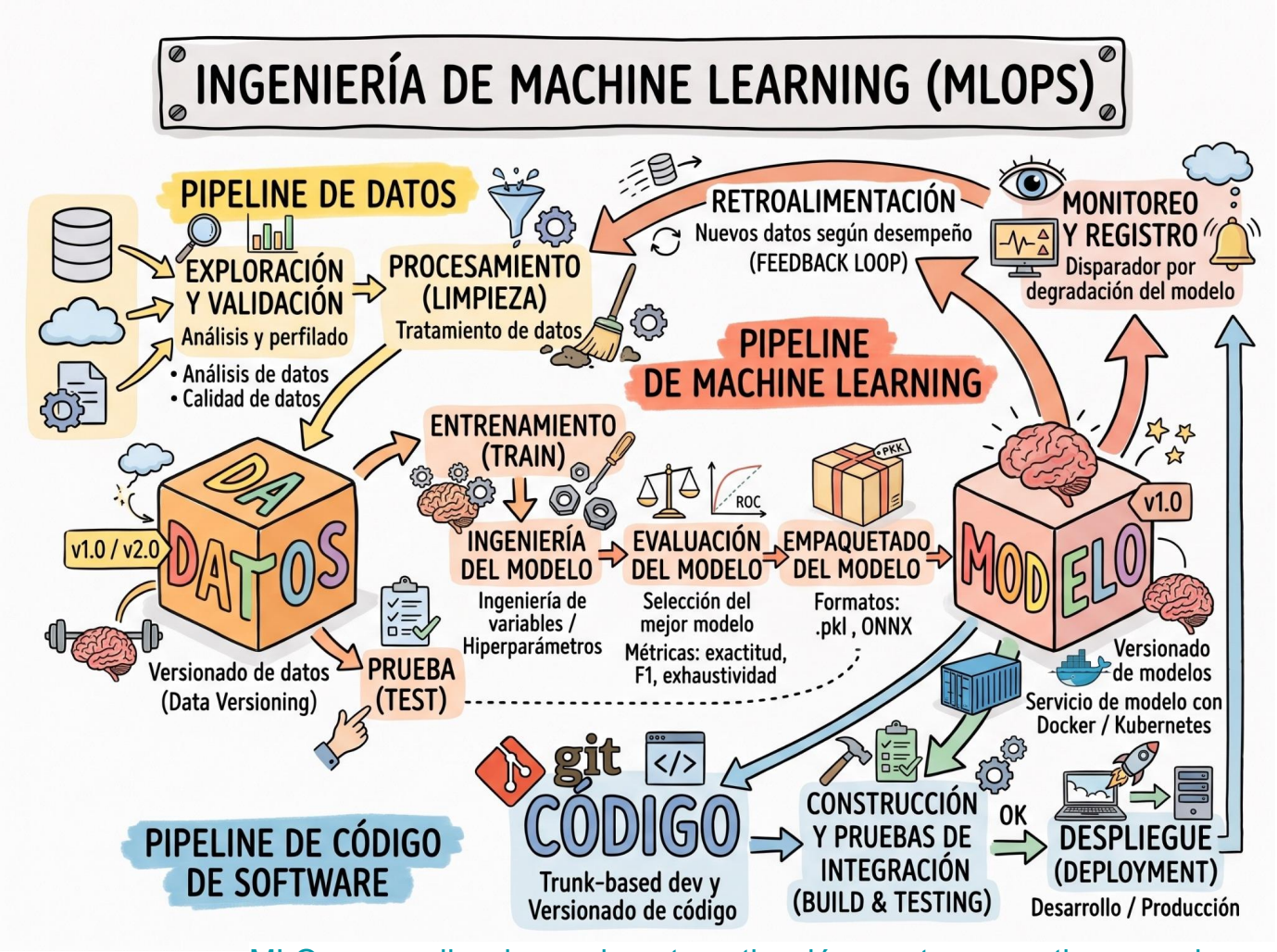
Reproducibilidad

Garantizar que cualquier miembro del equipo pueda obtener **el mismo** modelo con el mismo set de datos.



Monitoreo
(Seguimiento constante)

Vigilar el desempeño del modelo en **tiempo real** para detectar cuándo empieza a fallar.



MLOps: canalizaciones de automatización y entrega continua en el aprendizaje automático

INTRODUCCIÓN A GIT Y GITHUB



¿Qué es Git y GitHub?

Git (La Máquina del Tiempo)



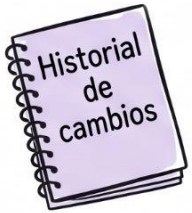
Herramienta que vive en tu computadora local.



Registra "fotos" (commits) de tu código a lo largo del tiempo.



Te permite regresar al pasado si algo falla.



Adiós a: "modelo_final_final_v2_este_si.ipynb".



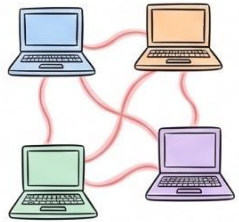
Sincronización (Push & Pull)

GitHub (La Nube y el Trabajo en Equipo)

Una plataforma en la nube (como Google Drive para desarrolladores).



Guarda de forma segura los proyectos de Git para que no se pierdan si se rompe tu máquina.



Permite a múltiples personas (como tu equipo) trabajar juntas en el mismo código sin pisarse el trabajo.



¿Qué es Git y GitHub?

Git (La Máquina del Tiempo)



Herramienta que vive en tu computadora local.



Registra "fotos" (commits) de tu código a lo largo del tiempo.



Te permite regresar al pasado si algo falla.



Adiós a: "modelo_final_final_v2_este_si.ipynb".



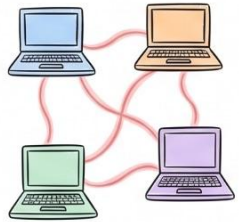
Sincronización (Push & Pull)

GitHub (La Nube y el Trabajo en Equipo)

Una plataforma en la nube (como Google Drive para desarrolladores).



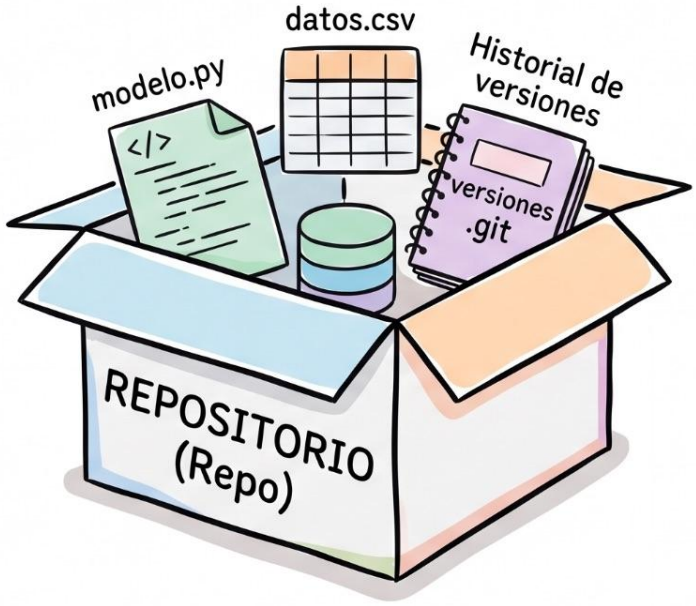
Guarda de forma segura los proyectos de Git para que no se pierdan si se rompe tu máquina.



Permite a múltiples personas (como tu equipo) trabajar juntas en el mismo código sin pisarse el trabajo.



¿Qué es un Repositorio (Repo)?



Es la carpeta oficial de tu proyecto. Guarda todo lo necesario: código, datos, documentación y la máquina del tiempo (historial).

El Espacio de Trabajo
(De tu laptop a la nube)

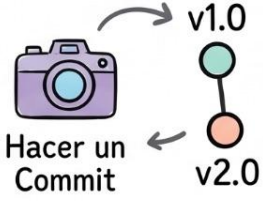
Git
(En la Nube)



1. Directorio de Trabajo (Local)

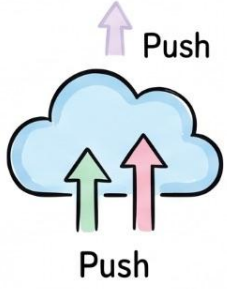
Donde escribes y editas tu código.

2. El Historial de Git
(Local .git)



Fotos o versiones guardadas en tu máquina.

3. GitHub
(En la Nube)

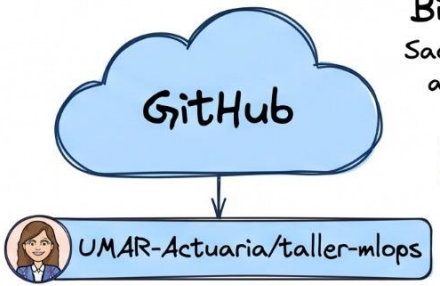


Tu proyecto visible y seguro en internet para tu equipo.

¿Para qué sirve? Centralizar el proyecto, no perder nada, colaborar con otros y automatizar pruebas.

¿Cómo trabajaremos en el Taller? (Fork y Clone)

1. Repo del Taller (Original)



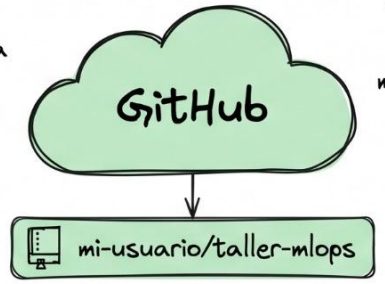
Este es el código de telecom original de la profesora. Como todos van a practicar, no pueden editar el mismo directo.

Bifurcar (FORK)

Saca una copia exacta a tu propia cuenta.

Saca una copia exacta a tu propia cuenta.

2. Tu Copia en la Nube (Tu Fork)



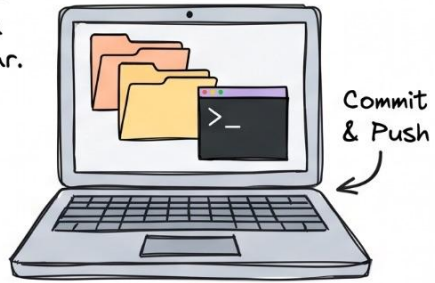
Es tu copia personal en internet. Aquí tienes total libertad para hacer pruebas, editar y equivocarte.

Clonar (CLONE)

Descarga todo a la máquina para trabajar.

Descarga todo a la máquina para trabajar.

3. Tu Computadora (Local)



Bajas el código de internet a la sala de cómputo para correr el modelo. Cuando termines, subes tus cambios ("Push") de regreso a tu cuenta.

Comando clave:

```
git clone https://github.com/usuario/taller-mlops.git
```

Introducción a las Ramas (Branches)



Rama de Desarrollo (feature-rama)

Una copia paralela de tu proyecto para experimentar. Creas tu u propia línea de tiempo.

Rama Principal (main / master)

Prueba de regresión

Ajuste de variables

Fusión (MERGE)

v1.0 (Código base)

v1.1 (Despliegue)

v1.2 (En producción)

Una vez que tus cambios en la rama alternativa funcionan a la perfección, se integran de regreso a la principal con total seguridad.

Es la versión del proyecto que funciona en el mundo real. Nunca se debe experimentar o romper aquí.



¿Por qué usarlas? Permiten probar nuevas ideas, hacer experimentos o corregir errores en equipo, sin el miedo de romper lo que ya está funcionando en producción.



Learn Git Branching